Software Design Description (SDD)

Fleet Management System

Architecture, database, modules and interface design

Document	Version	Status
Software Design Description (SDD)	1.0	Draft for review

1. Introduction

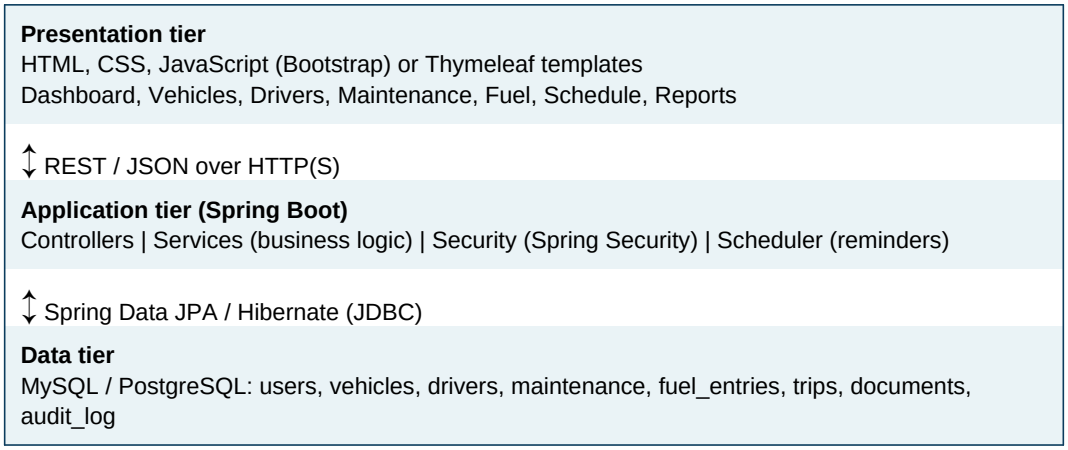
This document describes the design that implements the requirements in the FMS SRS: architecture, database, class and module design, API, key algorithms, user interface and testing.

1.1 Design Goals

- Clean layered Java design that is easy to extend and test.
- Reliable data handling with strong validation and conflict checks.
- Simple, responsive UI for managers and drivers.
- Easy deployment on a single server.

2. System Architecture

FMS uses a layered three-tier architecture following the MVC pattern: browser frontend, Spring Boot backend (controller, service and repository layers) and a relational database.



2.1 Technology Stack

Layer	Technology	Reason
Frontend	HTML5, CSS3, JavaScript, Bootstrap, Chart.js	Responsive UI, simple charts
Backend	Java 17, Spring Boot, Spring MVC	Industry-standard Java framework
Persistence	Spring Data JPA, Hibernate	Less boilerplate, object-relational mapping
Database	MySQL 8 / PostgreSQL	Reliable relational storage
Security	Spring Security, BCrypt, JWT/session	Role-based access
Build / Test	Maven, JUnit 5, Mockito, Postman	Build and automated testing
Reports	JasperReports / iText	PDF export

2.2 Package Structure

```
com.fms
  controller  (REST controllers)
  service     (business logic)
  repository (JPA repositories)
  model      (entities and enums)
  dto        (request/response objects)
  security    (auth, roles)
  scheduler   (reminder jobs)
```

```
exception   (global error handling)
```

3. Module and Class Design

Module	Responsibility	Main classes / methods
Authentication	Login, roles, tokens	AuthController, UserService, SecurityConfig
Vehicle	CRUD, status, documents	Vehicle, VehicleService.add(), updateStatus(), documentsExpiring()
Driver	CRUD, licence validity	Driver, DriverService.add(), isEligible()
Maintenance	Records, intervals, reminders	MaintenanceRecord, MaintenanceService.log(), nextDue(), overdueList()
Fuel	Entries, mileage, cost	FuelEntry, FuelService.record(), mileage(), costPerKm(), detectAnomaly()
Scheduling	Trips, conflict detection	Trip, TripService.create(), hasConflict(), start(), complete()
Reporting	Dashboard data, reports, export	ReportService.summary(), fuelReport(), exportPdf(), exportCsv()
Reminder Scheduler	Daily job for due items	ReminderJob.run() using @Scheduled

3.1 Core Entity Relationships

- A Vehicle has many MaintenanceRecords, FuelEntries, Documents and Trips.
- A Driver has many Trips and many FuelEntries.
- A Trip links exactly one Vehicle and one Driver.
- A User may be linked to one Driver profile (for the Driver role).

4. Database Design

Table	Columns (PK = primary key, FK = foreign key)
users	user_id (PK), username (unique), password_hash, role, driver_id (FK, nullable), is_active
vehicles	vehicle_id (PK), reg_no (unique), make, model, year, vehicle_type, fuel_type, tank_capacity, odometer, status, is_active
drivers	driver_id (PK), name, phone, licence_no (unique), licence_class, licence_expiry, status, is_active
vehicle_documents	doc_id (PK), vehicle_id (FK), doc_type, doc_number, expiry_date
maintenance_records	record_id (PK), vehicle_id (FK), type, service_date, odometer, vendor, description, cost, next_due_km, next_due_date
service_intervals	interval_id (PK), vehicle_id (FK), service_type, every_km, every_days
fuel_entries	fuel_id (PK), vehicle_id (FK), driver_id (FK), fuel_date, litres, price_per_litre, total_cost, odometer, station, anomaly_flag
trips	trip_id (PK), vehicle_id (FK), driver_id (FK), start_time, end_time, source, destination, purpose, start_odometer, end_odometer, status
audit_log	log_id (PK), user_id (FK), action, entity, entity_id, ts

Indexes: vehicles(reg_no), fuel_entries(vehicle_id, fuel_date), trips(driver_id, start_time), trips(vehicle_id, start_time). Vehicles and drivers use soft delete (is_active) to preserve history.

5. API Design

Method	Endpoint	Description
POST	/api/auth/login	Authenticate user
GET/POST	/api/vehicles	List or add vehicles
GET/PUT/DELETE	/api/vehicles/{id}	View, update or deactivate vehicle
GET/POST	/api/drivers	List or add drivers
GET/POST	/api/maintenance	List or log maintenance
GET	/api/maintenance/due	Upcoming and overdue maintenance
GET/POST	/api/fuel	List or record fuel entries
GET	/api/fuel/summary?vehicleId=&from;=&to;=	Mileage, cost per km, total cost
GET/POST	/api/trips	List or create trips (conflict check)
PUT	/api/trips/{id}/start, /complete	Update trip status and odometer
GET	/api/dashboard	Dashboard summary
GET	/api/reports/{type}?from=&to;=&format;=	Generate report (PDF/CSV)

Example fuel entry request:

```
{ "vehicleId": 4, "driverId": 2, "fuelDate": "2026-11-09", "litres": 38.5,
"pricePerLitre": 102.4, "odometer": 45210, "station": "HP Banjara Hills" }
```

6. Algorithm Design

6.1 Trip Conflict Detection

```
hasConflict(vehicle, driver, newStart, newEnd):
  if vehicle.status == UNDER_MAINTENANCE or INACTIVE: return true
  if driver.licence_expiry < newEnd: return true
  overlap = existing trips of vehicle OR driver where
    status in (PLANNED, ONGOING) and start < newEnd and end > newStart
  return overlap is not empty
```

6.2 Fuel Mileage and Anomaly Check

- Validate: odometer greater than previous entry and litres greater than 0.
- mileage = (odometer - previousOdometer) / litres, using full-tank-to-full-tank entries.
- totalCost = litres x pricePerLitre; costPerKm = totalCost / distance.
- Flag anomaly if litres exceed tank capacity or mileage is below 60% of the vehicle's average.

6.3 Maintenance Due Calculation

- After each service: next_due_km = odometer + every_km; next_due_date = service_date + every_days.
- A daily scheduled job marks items as DUE SOON (within 500 km or 7 days) or OVERDUE, and sends reminders.
- The same job checks vehicle document and driver licence expiry (30-day warning).

7. User Interface Design

Screen	Content
Login	Username, password, role-based redirect
Dashboard	Cards: total vehicles, on trip, under maintenance; monthly fuel and maintenance cost chart; due maintenance list; today's trips; expiring documents
Vehicles	Searchable table, add/edit form, vehicle detail with tabs: Info, Maintenance, Fuel, Trips, Documents
Drivers	Table with licence status, add/edit form, driver trip history
Maintenance	Log form, history table, due and overdue list
Fuel	Entry form, history table, mileage and cost-per-km chart
Schedule	Calendar or timeline view of trips, create trip form with conflict warnings
Reports	Select report, period, vehicle; preview, export PDF/CSV
Admin	User management, intervals, system settings

8. Security Design

- BCrypt password hashing; role-based endpoint protection with Spring Security.
- DTO validation (@Valid) and parameterised queries via JPA to prevent injection.
- Drivers can access only their own trips and fuel entries.
- Audit log for create, update and delete actions.

9. Error Handling and Logging

- @ControllerAdvice returns consistent JSON errors (400, 401, 403, 404, 409 for conflicts, 500).
- SLF4J/Logback logging with daily rotation; sensitive data never logged.

10. Test Plan

Level	What is tested	Tool
Unit	Mileage, cost and due-date calculations; conflict detection; validation	JUnit 5, Mockito
Integration	Controller to database flows for each module	Spring Boot Test, H2
System	Use cases UC-01 to UC-06 against acceptance criteria	Manual test cases, Postman
Security	Role access, invalid tokens, injection attempts	Manual checklist
Performance	Lists and dashboard with 10,000 records	Scripts / JMeter

11. Requirement Traceability

Requirement	Design element
FR-01, FR-17	Authentication module, users, audit_log
FR-02, FR-03	Vehicle module, vehicles, vehicle_documents
FR-04	Driver module, drivers
FR-05, FR-06, FR-07	Maintenance module, service_intervals, ReminderJob
FR-08, FR-09, FR-10	Fuel module, fuel_entries

Requirement	Design element
FR-11, FR-12, FR-13	Scheduling module, trips, hasConflict()
FR-14, FR-15, FR-16	Reporting module, dashboard API, list screens

12. Deployment and Future Enhancements

- Deployment: executable Spring Boot JAR with Nginx and a managed database, or Docker Compose.
- Future: GPS tracking, mobile app for drivers, fuel-card integration, predictive maintenance, route optimisation.